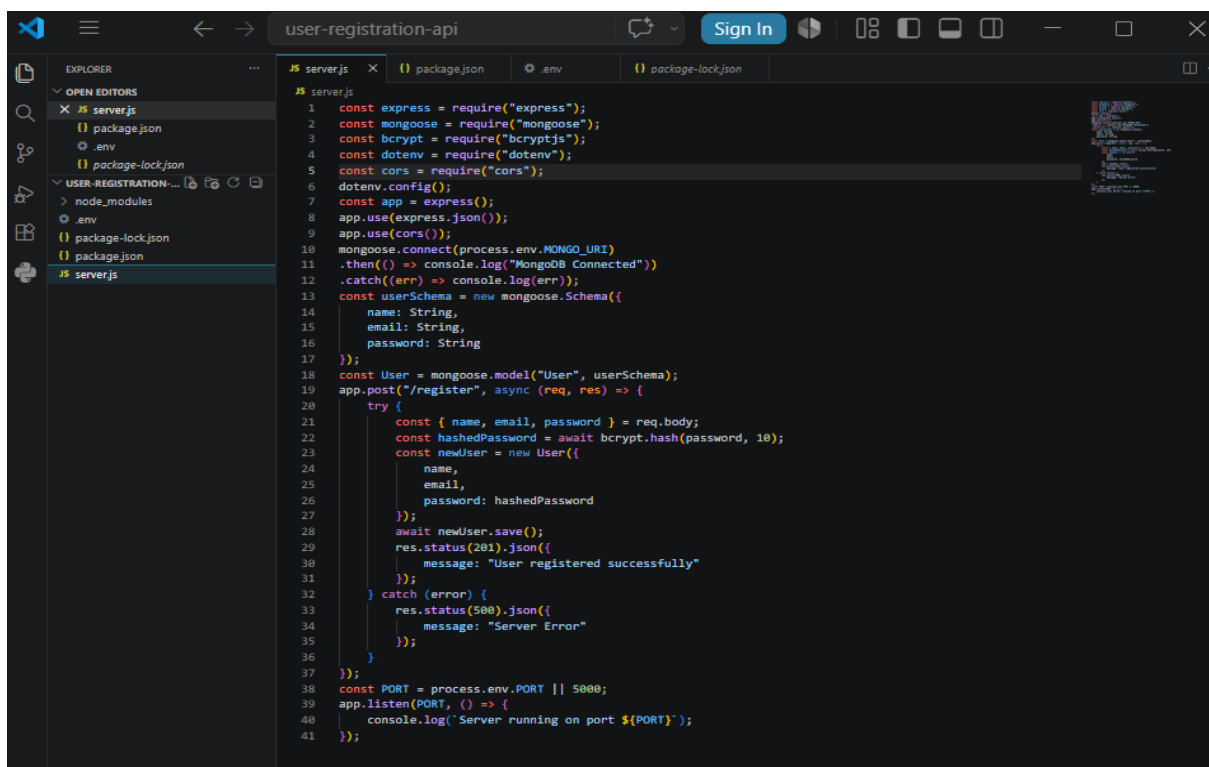


DAY-4 BACKEND TASKS

Nainisha Bandari

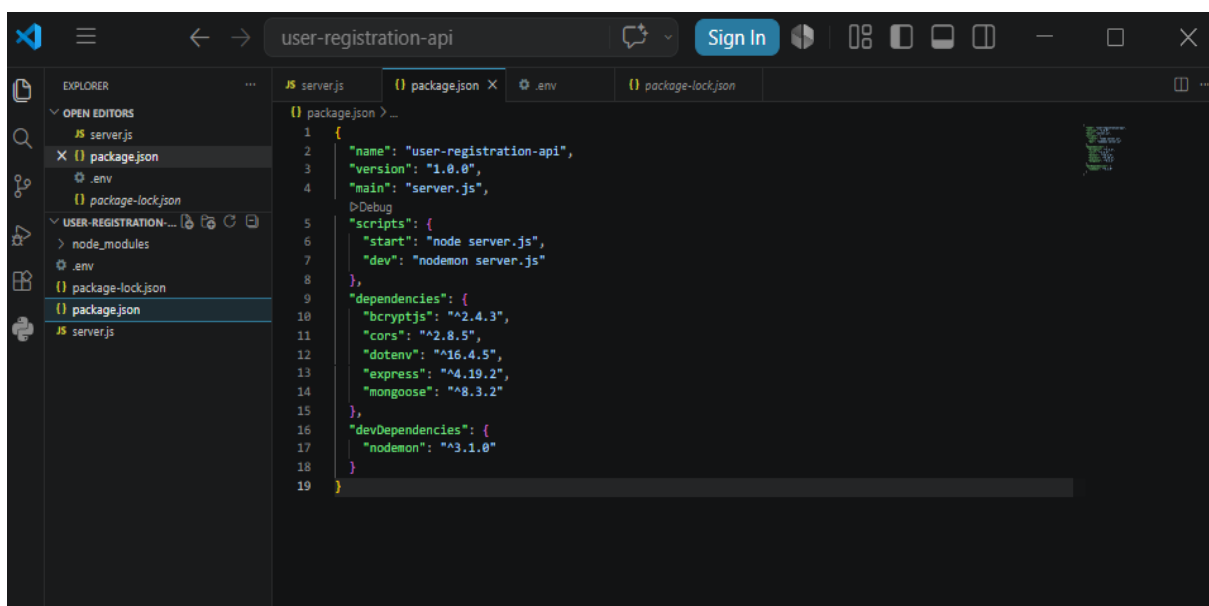
Nainishabandari1111@gmail.com

TASK1-User Registration API



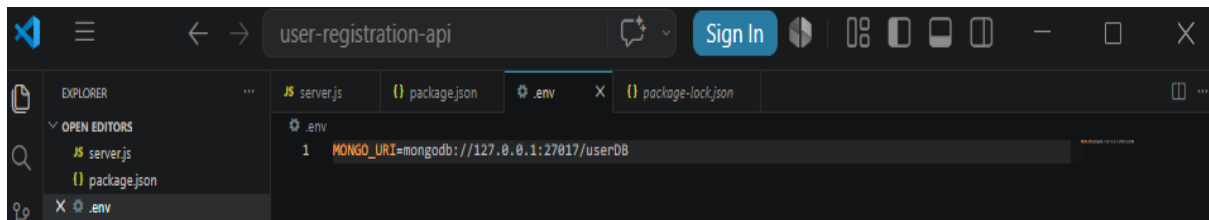
The screenshot shows the VS Code editor with the 'user-registration-api' project open. The Explorer sidebar on the left shows the file structure: 'server.js', 'package.json', '.env', and 'package-lock.json'. The main editor displays the 'server.js' file with the following code:

```
1 const express = require("express");
2 const mongoose = require("mongoose");
3 const bcrypt = require("bcryptjs");
4 const dotenv = require("dotenv");
5 const cors = require("cors");
6 dotenv.config();
7 const app = express();
8 app.use(express.json());
9 app.use(cors());
10 mongoose.connect(process.env.MONGO_URI)
11 .then(() => console.log("MongoDB Connected"))
12 .catch((err) => console.log(err));
13 const userSchema = new mongoose.Schema({
14   name: String,
15   email: String,
16   password: String
17 });
18 const User = mongoose.model("User", userSchema);
19 app.post("/register", async (req, res) => {
20   try {
21     const { name, email, password } = req.body;
22     const hashedPassword = await bcrypt.hash(password, 10);
23     const newUser = new User({
24       name,
25       email,
26       password: hashedPassword
27     });
28     await newUser.save();
29     res.status(201).json({
30       message: "User registered successfully"
31     });
32   } catch (error) {
33     res.status(500).json({
34       message: "Server Error"
35     });
36   }
37 });
38 const PORT = process.env.PORT || 5000;
39 app.listen(PORT, () => {
40   console.log(`Server running on port ${PORT}`);
41 });
```

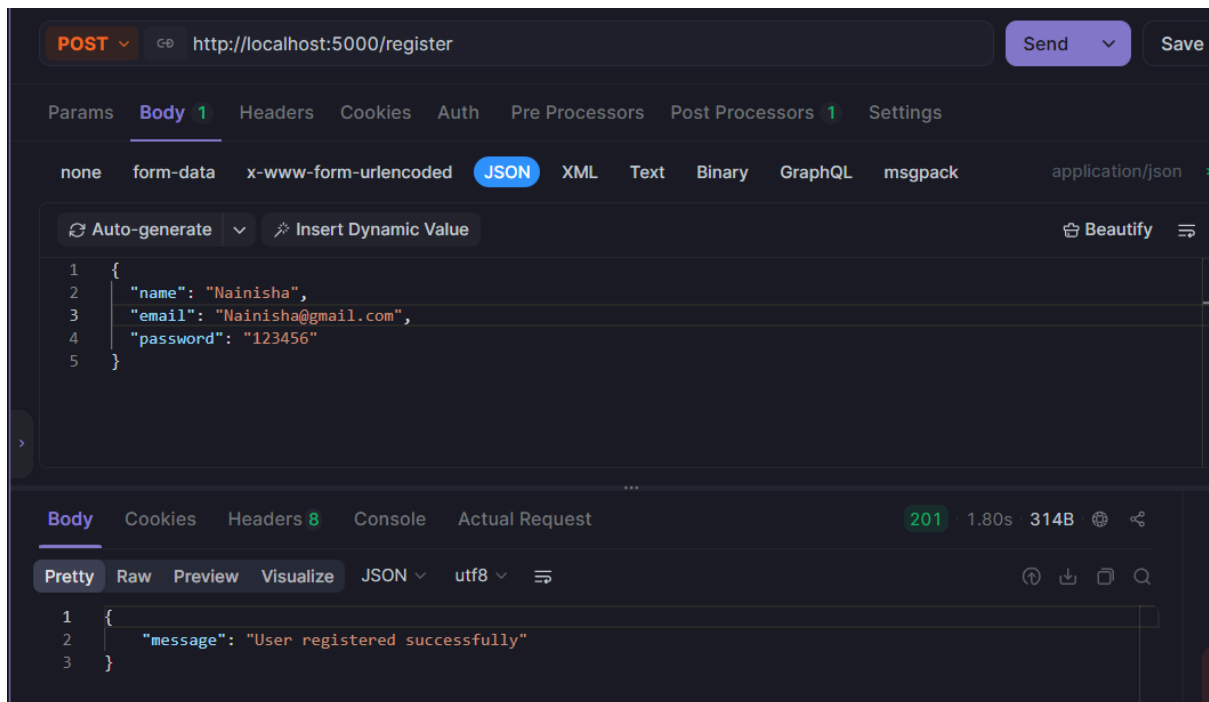


The screenshot shows the VS Code editor with the 'user-registration-api' project open. The Explorer sidebar on the left shows the file structure: 'server.js', 'package.json', '.env', and 'package-lock.json'. The main editor displays the 'package.json' file with the following code:

```
1 {
2   "name": "user-registration-api",
3   "version": "1.0.0",
4   "main": "server.js",
5   "scripts": {
6     "start": "node server.js",
7     "dev": "nodemon server.js"
8   },
9   "dependencies": {
10    "bcryptjs": "^2.4.3",
11    "cors": "^2.8.5",
12    "dotenv": "^16.4.5",
13    "express": "^4.19.2",
14    "mongoose": "^8.3.2"
15  },
16  "devDependencies": {
17    "nodemon": "^3.1.0"
18  }
19 }
```

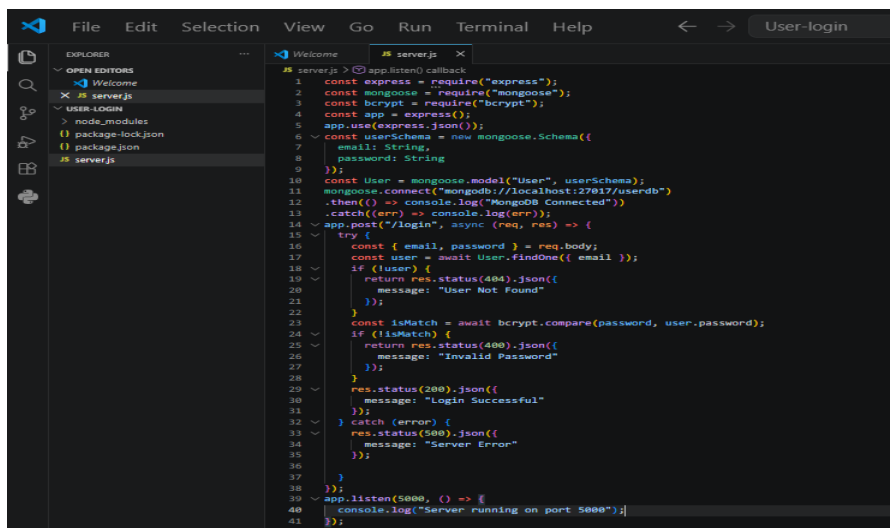


POST/Register API

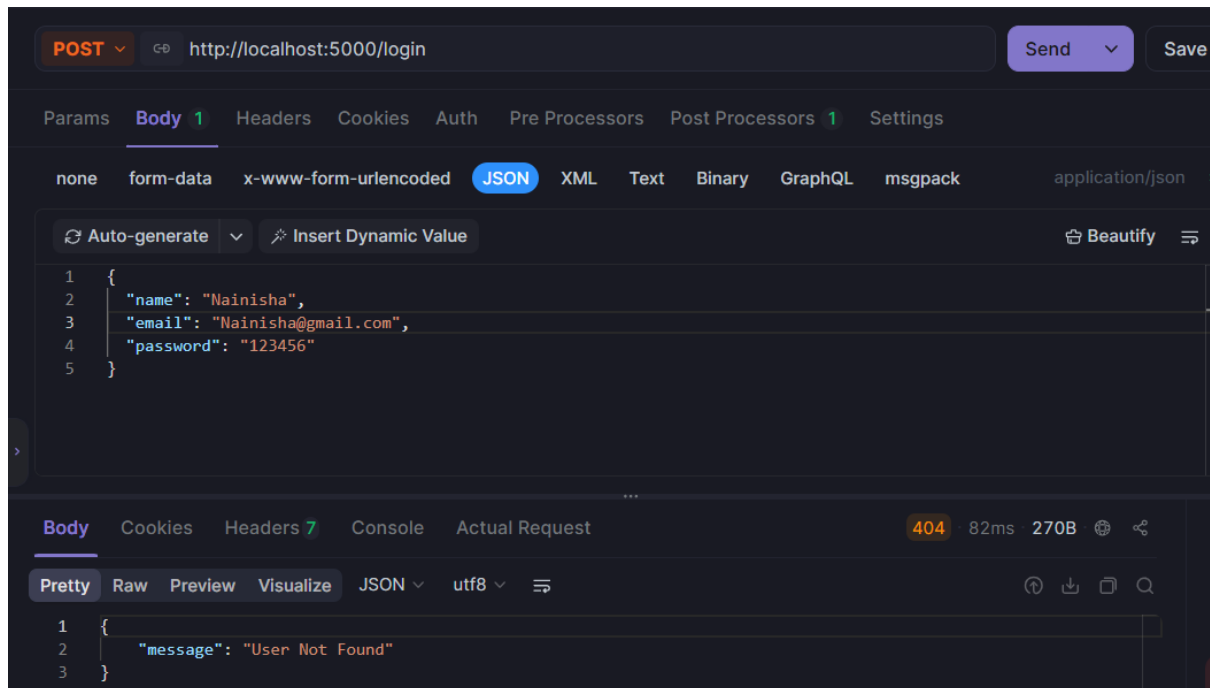


TASK2-User Login API

Step1:Add Login API



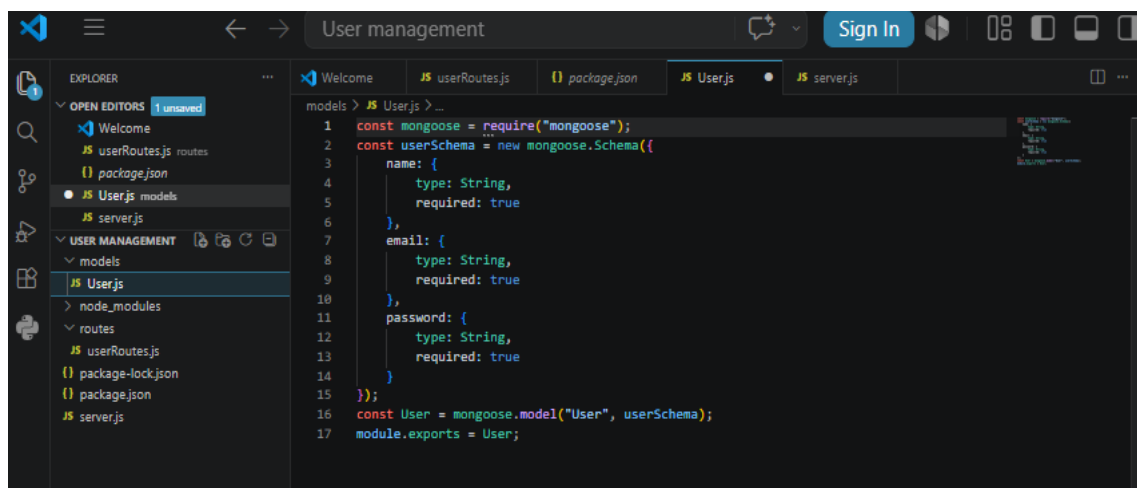
2.Test login API

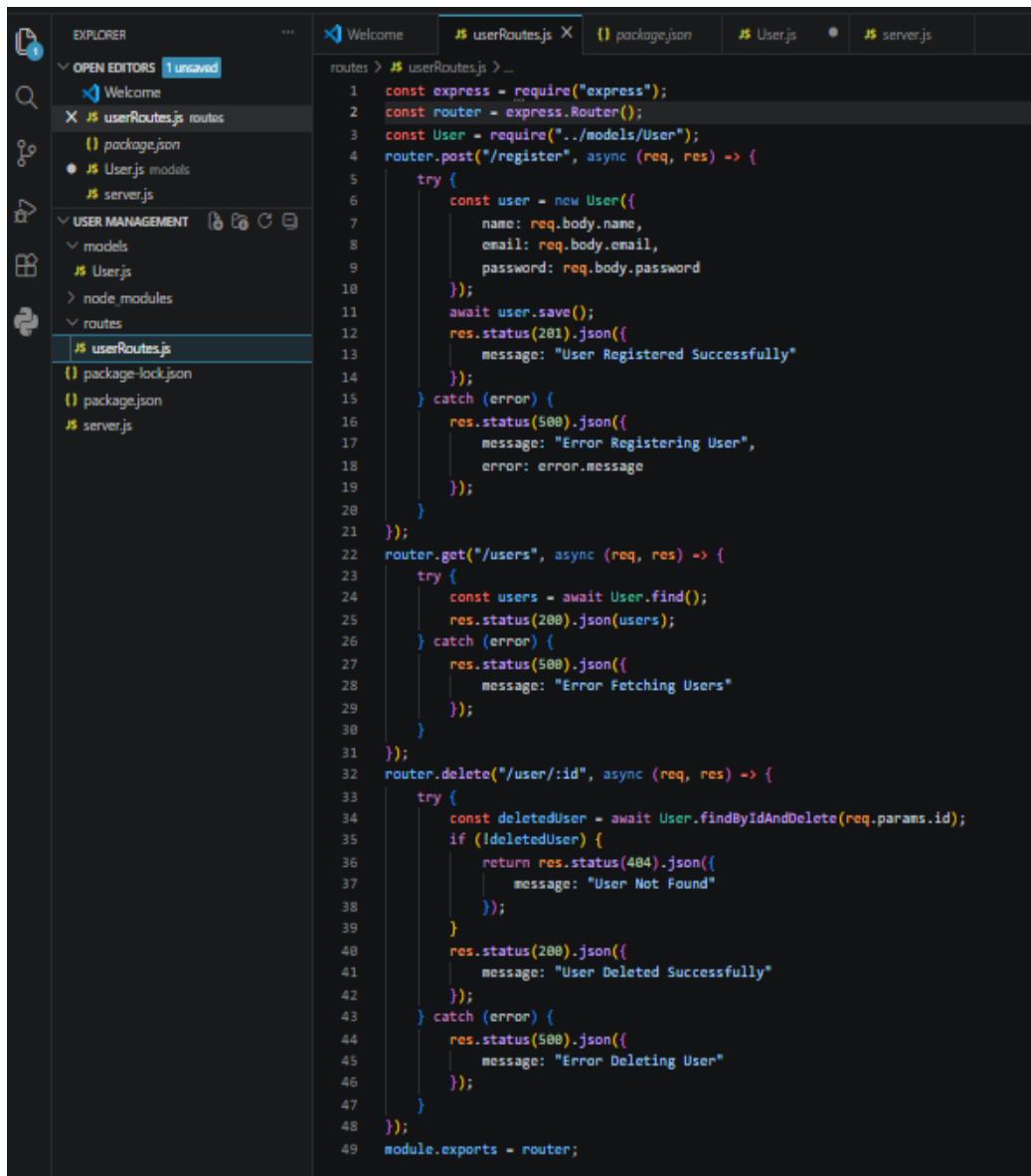


Authentication is working successfully.

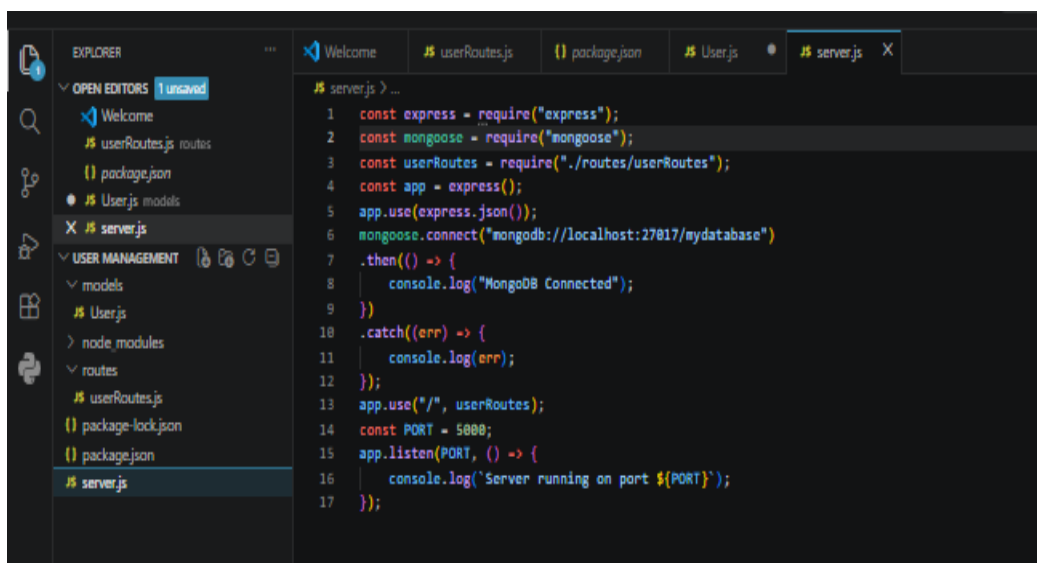
TASK3-User Management APIs

Step 1: Write codes in User.js, userRoutes.js, server.js



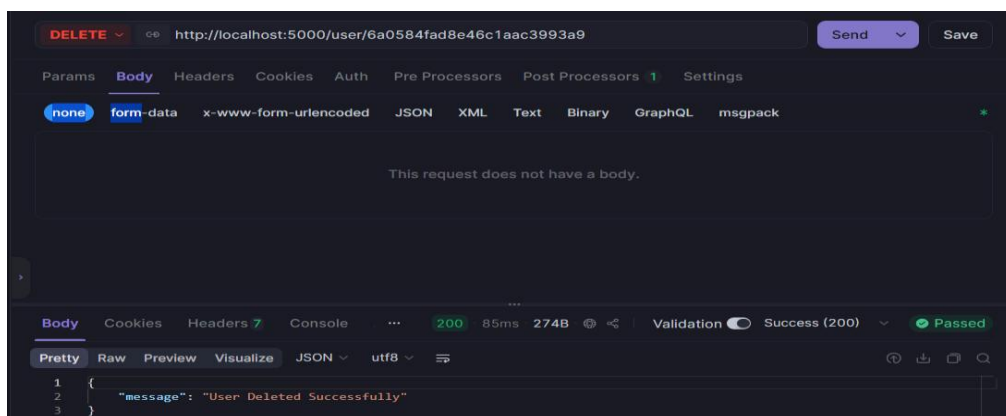
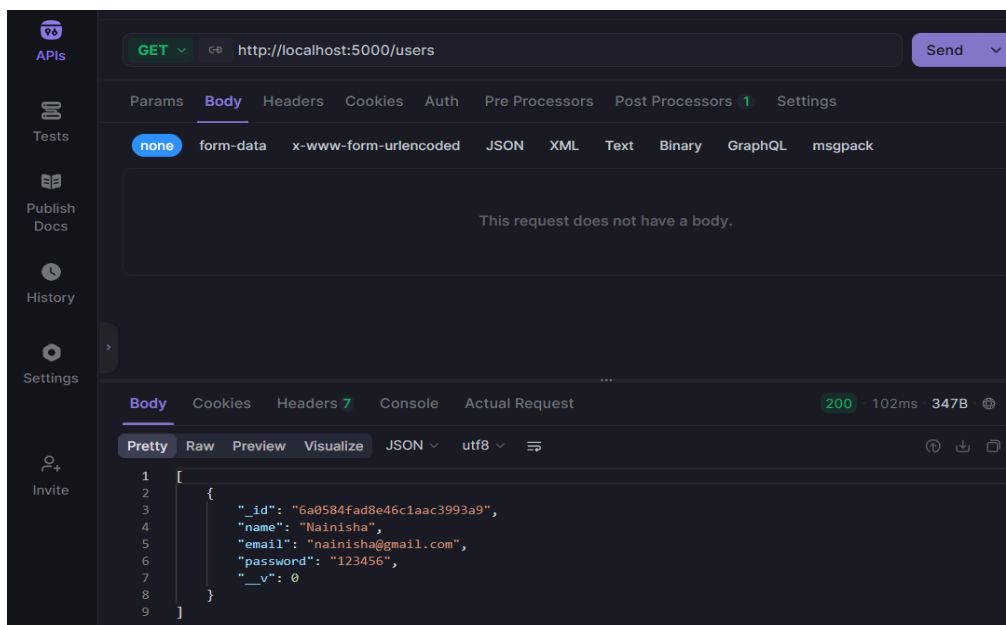
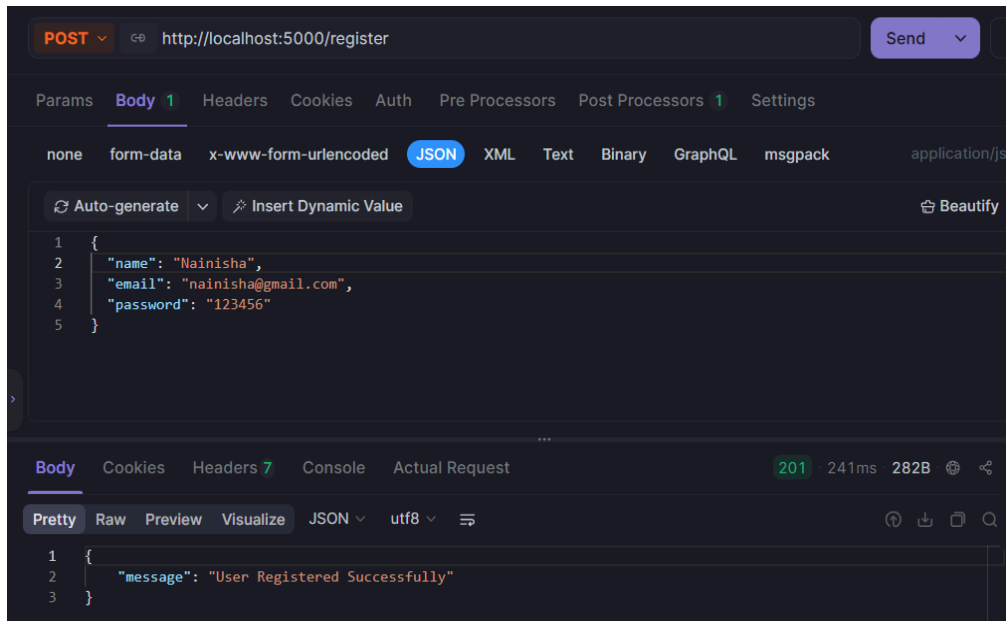


```
routes > # userRoutes.js > ...
1  const express = require("express");
2  const router = express.Router();
3  const User = require("../models/User");
4  router.post("/register", async (req, res) => {
5    try {
6      const user = new User({
7        name: req.body.name,
8        email: req.body.email,
9        password: req.body.password
10     });
11     await user.save();
12     res.status(201).json({
13       message: "User Registered Successfully"
14     });
15   } catch (error) {
16     res.status(500).json({
17       message: "Error Registering User",
18       error: error.message
19     });
20   }
21 });
22 router.get("/users", async (req, res) => {
23   try {
24     const users = await User.find();
25     res.status(200).json(users);
26   } catch (error) {
27     res.status(500).json({
28       message: "Error Fetching Users"
29     });
30   }
31 });
32 router.delete("/user/:id", async (req, res) => {
33   try {
34     const deletedUser = await User.findByIdAndDelete(req.params.id);
35     if (!deletedUser) {
36       return res.status(404).json({
37         message: "User Not Found"
38       });
39     }
40     res.status(200).json({
41       message: "User Deleted Successfully"
42     });
43   } catch (error) {
44     res.status(500).json({
45       message: "Error Deleting User"
46     });
47   }
48 });
49 module.exports = router;
```

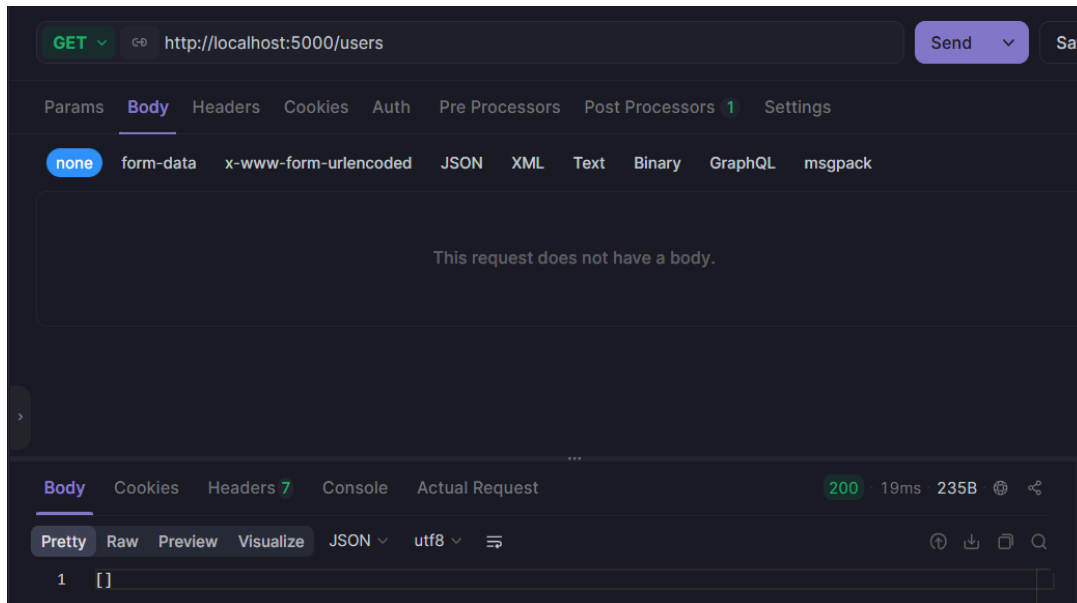


```
# server.js > ...
1  const express = require("express");
2  const mongoose = require("mongoose");
3  const userRoutes = require("./routes/userRoutes");
4  const app = express();
5  app.use(express.json());
6  mongoose.connect("mongodb://localhost:27017/mydatabase")
7    .then(() => {
8      console.log("MongoDB Connected");
9    })
10   .catch((err) => {
11     console.log(err);
12   });
13 app.use("/", userRoutes);
14 const PORT = 5000;
15 app.listen(PORT, () => {
16   console.log(`Server running on port ${PORT}`);
17 });
```

STEP 2 — Test POST,GET,DELETE API

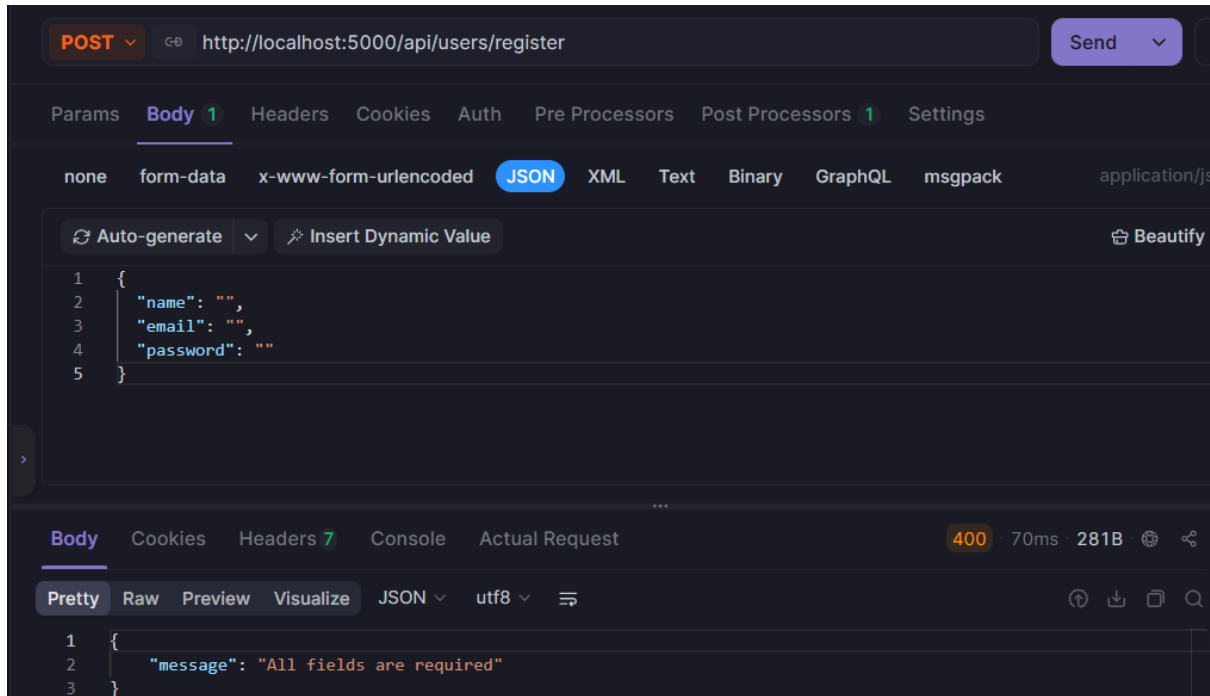


Step 3:Verify Deletion

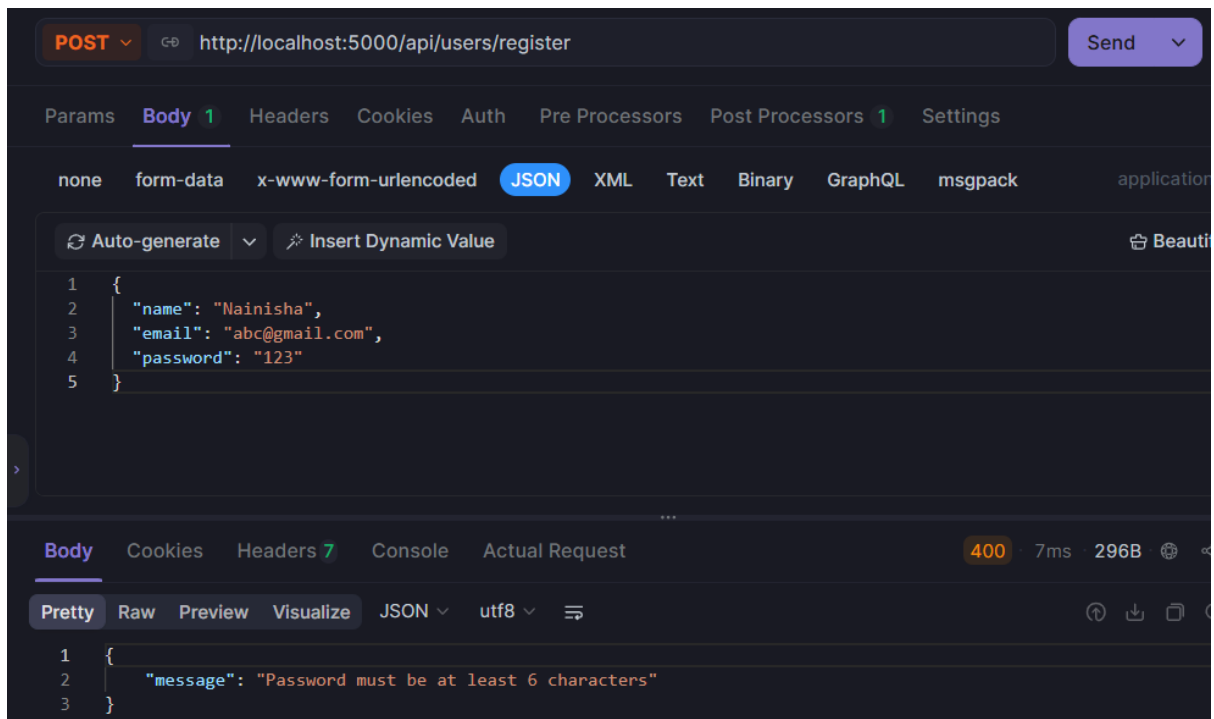


Task 4:VALIDATION AND SECURITY

1.Validation for empty fields



2.Short Password



3.Duplicate Email Registration

